

Cours 3: Manipulation de Data Sets en SAS

Luis Alberto Gomez

Master 1 SE et DS2E

Année 2025 – 2026

Plan de la séance

- 1 Introduction aux Data Sets
- 2 Manipulation des variables
- 3 Filtrage des observations
- 4 Conditions et logique
- 5 Gestion des variables
- 6 Combinaison de bases de données
- 7 Fusion de bases de données : MERGE

Le DATA STEP

Le DATA STEP permet de créer ou modifier un data set.

```
data nouveau\_dataset;  
    set ancien\_dataset;  
run;
```

Chaque observation est lue séquentiellement et les instructions sont appliquées ligne par ligne.

Créer de nouvelles variables

```
data voitures2;  
    set voitures;  
  
    prix_ttc = prix * 1.20;  
    age = 2024 - annee;  
run;
```

Les variables peuvent être numériques ou caractères.

Sélectionner ou supprimer des variables

Garder certaines variables

```
data voitures2;  
    set voitures;  
    keep marque prix annee;  
run;
```

Supprimer certaines variables

```
data voitures2;  
    set voitures;  
    drop couleur proprietaire;  
run;
```

Ajouter des attributs aux variables

On peut ajouter plusieurs attributs :

- Label
- Format

```
label prix = "Prix du vehicule";  
format prix euro10.;
```

Instruction LABEL

```
data voitures2;  
    set voitures;  
  
    label prix = "Prix du vehicule"  
          annee = "Annee de fabrication";  
run;
```

Les labels permettent d'afficher des descriptions plus lisibles.

Filtrer des observations avec IF

```
data voitures2;  
    set voitures;  
  
    if prix > 20000;  
run;
```

Seules les observations qui satisfont la condition sont conservées.

Utiliser THEN DELETE

```
data voitures2;  
    set voitures;  
  
    if prix < 5000 then delete;  
run;
```

Toutes les observations qui vérifient la condition sont supprimées.

Différence entre IF et WHERE

WHERE : filtre avant lecture

```
data voitures2;  
    set voitures;  
    where prix > 20000;  
run;
```

IF : filtre après lecture

```
data voitures2;  
    set voitures;  
  
    if prix > 20000;  
run;
```

Instruction IF / ELSE IF

```
data voitures2;  
  set voitures;  
  
  if prix < 10000 then categorie="Bas";  
  else if prix < 20000 then categorie="Moyen";  
  else categorie="Haut";  
run;
```

Permet de créer des variables conditionnelles.

Utiliser THEN OUTPUT

```
data voitures_cheres voitures_pas_cheres;  
  set voitures;  
  
  if prix > 20000 then output voitures_cheres;  
  else output voitures_pas_cheres;  
run;
```

Permet d'envoyer les observations dans différents data sets.

Renommer des variables

```
data voitures2;  
    set voitures;  
  
    rename prix = prix_euro;  
run;
```

Ou directement dans SET :

```
set voitures(rename=(prix=prix_euro));
```

Les formats permettent de modifier l'affichage.

```
data voitures2;  
    set voitures;  
  
    format prix euro10.;  
run;
```

Exemples :

- date9.
- euro10.
- dollar10.

Concaténer des data sets

La concaténation consiste à empiler les observations.

```
data total;  
    set data1 data2 data3;  
run;
```

Conditions :

- Les variables doivent avoir les mêmes noms
- Les types doivent être compatibles

Fusion de bases de données avec MERGE

Principe du MERGE

Le MERGE permet de combiner plusieurs data sets en utilisant une ou plusieurs variables communes.

```
data fusion;  
    merge data1 data2;  
    by id;  
run;
```

Conditions importantes :

- Les data sets doivent être triés
- La variable du BY doit exister dans chaque data set
- Le résultat dépend de la relation entre les observations

Trier les données avant un MERGE

Avant d'utiliser MERGE, il est indispensable de trier les data sets.

```
proc sort data=data1;  
    by id;  
run;  
  
proc sort data=data2;  
    by id;  
run;
```

Ensuite seulement :

```
data fusion;  
    merge data1 data2;  
    by id;  
run;
```

Relations possibles entre les données

La relation entre les observations peut être de plusieurs types.

- 1 One-to-one
- 2 One-to-many
- 3 Many-to-many
- 4 One or many to missing

Chaque relation peut produire un résultat différent lors du merge.

Merge One-to-One

Chaque observation dans data1 correspond à une seule observation dans data2.

```
data data1;  
input id age;  
datalines;  
1 25  
2 30  
3 40  
;  
run;  
  
data data2;  
input id salaire;  
datalines;  
1 2000  
2 2500  
3 3000  
;  
run;
```

Résultat du One-to-One

```
data fusion;  
    merge data1 data2;  
    by id;  
run;
```

Résultat :

id	age	salaire
1	25	2000
2	30	2500
3	40	3000

Chaque observation est combinée avec sa correspondance.

Merge One-to-Many

Une observation dans un data set correspond à plusieurs observations dans l'autre.

Exemple :

- data1 : individus
- data2 : achats

```
data fusion;  
    merge individus achats;  
    by id;  
run;
```

Résultat du One-to-Many

Exemple :

id	age	achat
1	25	livre
1	25	stylo
2	30	ordinateur

Les informations du premier data set sont répétées pour chaque observation correspondante.

Merge Many-to-Many

Une clé apparaît plusieurs fois dans les deux data sets.

- plusieurs observations dans data1
- plusieurs observations dans data2

Dans ce cas :

- SAS associe les observations dans l'ordre

Ce type de merge peut produire des résultats inattendus.

Observations sans correspondance

Certaines observations peuvent exister dans un seul data set.

Exemple :

data1	data2
id=1	id=1
id=2	id=3

Après le merge :

id	var1	var2
1	...	...
2	...	.
3	.	...

Les valeurs manquantes sont représentées par .

Contrôler le MERGE avec IN=

On peut identifier l'origine des observations avec l'option IN=.

```
data fusion;  
merge data1(in=a) data2(in=b);  
by id;  
  
if a and b then output;  
run;
```

Les variables temporaires permettent de contrôler le merge.

- a and b : observations présentes dans les deux tables
- a : observations seulement dans data1
- b : observations seulement dans data2

Cela permet de reproduire différents types de jointures.

Toujours :

- Trier les data sets avec PROC SORT
- Vérifier l'unicité de la clé
- Comprendre la relation entre les tables

Attention au cas **many-to-many** qui peut produire des résultats inattendus.